RL-BPINN

超车决策系统

模块化拆解与系统架构讲解

Modular System Breakdown & Architecture Guide

文档版本	v2.0 (模块化拆解版)
系统模块	8 个独立模块 + 1 个调度器
核心技术	BPINN / MC Dropout / RL / Physics Constraint / PD Controller
源码文件	18 Python + 1 HTML + 1 EXE
生成日期	2026-08-05

目录

1. 系统总览 -- 模块化架构与设计哲学

2. Module 1: 场景与数据 (Scenario & Data)

3. Module 2: 传感器窗口 (Sensor Window)

4. Module 3: 前向 BPINN (Forward BPINN)

5. Module 4: 逆向 BPINN (Inverse BPINN)

6. Module 5: 不确定性引擎 (Uncertainty Engine)

7. Module 6: RL 决策引擎 (RL Decision Engine)

8. Module 7: 奖励系统 (Reward System)

9. Module 8: 运动控制器 (Motion Controller)

10. 滚动窗口调度器 (Rolling Window Scheduler)

11. 模块接口与数据契约 (Module Interfaces)

12. 强化学习形式化定义 (RL Formulation)

13. 源码文件映射 (Code Mapping)

1. 系统总览

RL-BPINN 超车系统是一个将贝叶斯物理信息神经网络 (BPINN) 与强化学习 (RL) 深度融合的自动驾驶超车决策框架。系统被拆解为 8 个高内聚、低耦合的模块，每个模块负责一个明确的功能域，通过标准化的数据契约进行交互。

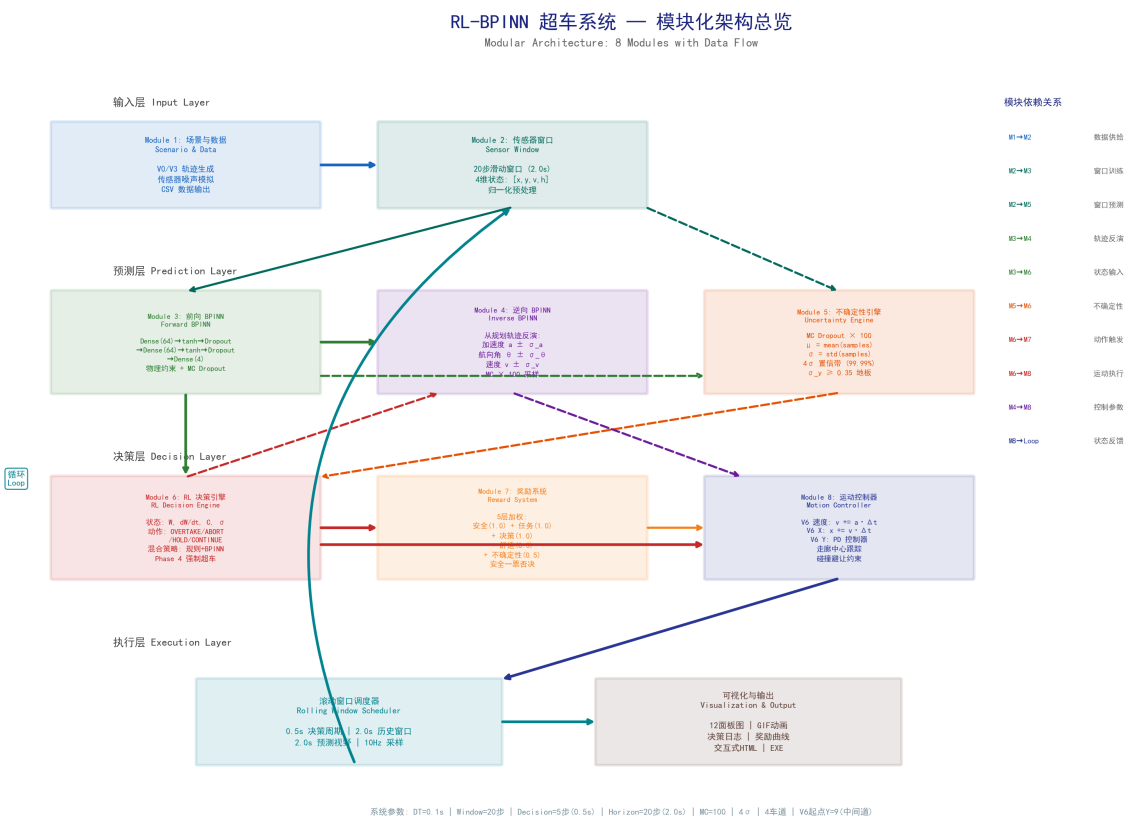


图 1-1: RL-BPINN 模块化架构总览 (8 模块 + 调度器)

1.1 模块划分原则

高内聚: 每个模块封装一个完整功能域 (预测、决策、控制等)，内部实现细节对其他模块不可见。

低耦合: 模块间通过明确定义的数据结构交互，无隐式依赖。任意模块可独立替换 (如 NumPy BPINN 替换为 JAX BPINN)。

可测试: 每个模块可独立运行和验证，输入输出格式固定，便于单元测试。

可扩展: 新增模块 (如 V2X 通信模块) 只需实现标准接口，无需修改现有模块。

1.2 模块概览表

模块	名称	功能	核心文件	代码行数
M1	场景与数据	V0/V3 轨迹生成, 噪声模拟	rolling_window_scenario.py, bpinn_complete_overtaking.py	~150行
M2	传感器窗口	20步滑动窗口, 归一化	rolling_window_bpinn_simulation.py	~50行
M3	前向 BPINN	轨迹预测 + 不确定性	ForwardBPINN, LightweightBPINN classes	~280行
M4	逆向 BPINN	控制参数反演	InverseBPINN class	~30行

模块	名称	功能	核心文件	代码行数
M5	不确定性引擎	MC Dropout, 4sigma置信带	predict_with_uncertainty()	~50行
M6	RL 决策引擎	状态感知, 策略, 动作	run_simulation() inner loop	~60行
M7	奖励系统	5 层加权奖励	compute_reward()	~50行
M8	运动控制器	V6 速度/位置/Y 更新	executeAction() / movement loop	~80行

2. Module 1: 场景与数据

职责: 生成 V0 (红色前车) 和 V3 (蓝色参考车) 的地面真值轨迹，模拟传感器噪声，输出标准化的时间序列数据。

2.1 输入 / 输出

方向	数据	格式	说明
输入	配置参数	全局常量	DT=0.1s, T_TOTAL=10s, N=101
输出	时间序列	t[101]	arange(0, 10.0, 0.1)
输出	V0 状态	v0_x[101], v0_y[101], v0_speed[101], v0_h[101]	4维 x 101步
输出	V3 状态	v3_x[101], v3_y[101], v3_speed[101], v3_h[101]	4维 x 101步
输出	噪声	sigma_x=0.05m, sigma_y=0.03m	高斯白噪声
输出	CSV 文件	bpinn_overtaking_complete_scenario.csv	303行 (101步 x 3车)

2.2 场景设计 -- 五阶段 Y 间隙动态

V0 和 V3 均以 25 m/s 匀速前进，Y 方向位置通过 smoothstep 函数实现"先变大-变小-再变大"的动态间隙模式。这一设计模拟真实高速公路中前车轻微偏移导致的超车窗口开合。

2.3 核心代码

```
def generate_ground_truth():
    t = np.arange(0, T_TOTAL + DT, DT)
    N = len(t)
    v0_speed = np.full(N, 25.0) # V0: constant 25 m/s
    v3_speed = np.full(N, 25.0) # V3: constant 25 m/s

    def v0_y_profile(ti):
        s1 = smoothstep(ti, 2.0, 4.0) # 11 -> 12.5
        s2 = smoothstep(ti, 4.0, 6.0) # 12.5 -> 10.5
        s3 = smoothstep(ti, 6.0, 8.0) # 10.5 -> 12.5
```

```
    y = 11.0
    y = y*(1-s1) + 12.5*s1
    y = y*(1-s2) + 10.5*s2
    y = y*(1-s3) + 12.5*s3
    return y
    # Y-gap: 4m -> 7m -> 3m -> 7m (open-close-reopen)
```

2.4 等效传感器规格

参数	值	说明
采样频率	10 Hz	DT=0.1s, 每秒10帧位置更新
决策频率	2 Hz	每0.5s一次RL决策 (5步)
历史窗口	2.0 s	20步历史数据用于BPINN训练
预测视野	2.0 s	20步未来轨迹预测
有效探测距离	~50 m	2.0s预测 x 25m/s车速
X位置精度	+/- 5 cm	sigma_x = 0.05m
Y位置精度	+/- 3 cm	sigma_y = 0.03m
车道宽度	4.0 m	4车道, 中心Y=3/7/11/15
车辆尺寸	4.5 x 1.8 m	CAR_LENGTH x CAR_WIDTH

3. Module 2: 传感器窗口

职责: 从全局时间序列中提取滑动窗口数据，进行归一化预处理，为 BPINN 提供标准输入。

3.1 滑动窗口机制

窗口大小为 20 步 (2.0s)，每 5 步 (0.5s) 向前滑动一次。在每个决策点 step_idx，提取 [step_idx - 20, step_idx] 范围内的 V0/V3 轨迹数据。

```
# Extract sliding window at each decision point
for step_idx in decision_indices: # [20, 25, 30, ..., 95]
w_start = max(0, step_idx - WINDOW_SIZE)
w_end = step_idx

t_window = t_gt[w_start:w_end] # [20]
v0_window = (v0_x[w_start:w_end], # [20]
v0_y[w_start:w_end], # [20]
v0_speed[w_start:w_end], # [20]
v0_h[w_start:w_end]) # [20]
v3_window = (v3_x[w_start:w_end], v3_y[w_start:w_end],
v3_speed[w_start:w_end], v3_h[w_start:w_end])
```

3.2 归一化预处理

各维度独立归一化: t_norm = (t - mean) / std。归一化参数在训练时计算并保存，推理时复用同一参数进行反归一化。

4. Module 3: 前向 BPINN

职责: 从窗口数据中学习车辆运动模式，预测未来 2.0 秒的 [x, y, v, heading] 四维状态，并通过 MC Dropout 量化预测不确定性。

4.1 网络架构

层	类型	维度	激活	Dropout
Input	时间 t	1	--	--
Hidden 1	Dense	64	tanh	p=0.05 (inverted)
Hidden 2	Dense	64	tanh	p=0.05 (inverted)
Output	Dense	4	--	--

4.2 损失函数 -- 数据 + 物理

$$L_{total} = L_{data} + w_{phys} * L_{physics}$$

L_data = MSE(pred, true) -- 4 维均方误差

L_physics = (dx/dt - v*cos(h))^2 + (dy/dt - v*sin(h))^2 -- 运动学约束

4.3 训练算法 -- Adam 手动反向传播

```
# Adam optimizer state
m_adam = {k: np.zeros_like(v) for k, v in self.params.items()}
v_adam = {k: np.zeros_like(v) for k, v in self.params.items()}
beta1, beta2, epsilon = 0.9, 0.999, 1e-8

for epoch in range(self.n_epochs): # 500 epochs
mask = self._gen_dropout_mask(training=True)
pred = self._forward(self.params, t_n, training=True, dropout_mask=mask)
error = pred - y_n # [N, 4]

# Manual backpropagation
grad_W3 = a2_d.T @ error
delta2 = (error @ W3.T) * (1 - a2**2) * mask['m2']
grad_W2 = a1_d.T @ delta2
delta1 = (delta2 @ W2.T) * (1 - a1**2) * mask['m1']
grad_W1 = t_n.T @ delta1

# Physics gradient (simplified)
if self.physics_weight > 0:
dx_dt = np.gradient(pred[:, 0], dt_val)
phys_err = (dx_dt - v_pred * np.cos(h_pred))**2
grad_W3 += self.physics_weight * 0.01 * a2_d.T @ phys_err

# Adam update
for key in self.params:
```

```
m_adam[key] = beta1*m_adam[key] + (1-beta1)*grads[key]
v_adam[key] = beta2*v_adam[key] + (1-beta2)*grads[key]**2
m_hat = m_adam[key] / (1 - beta1**(epoch+1))
v_hat = v_adam[key] / (1 - beta2**(epoch+1))
self.params[key] -= lr * m_hat / (np.sqrt(v_hat) + epsilon)
```

4.4 两种实现对比

特性	ForwardBPINN (NumPy)	LightweightBPINN	BPINN (JAX/Flax)
隐藏层	64	32	可调 (32~128)
Dropout	0.05	0.10	Optuna (0.05~0.3)
Epochs	500	300	8000 / 2000调优
MC	100	50	100
物理权重	0.5	1.0	Optuna (0.1~5.0)
学习率	0.005	0.01	Optuna (1e-4~5e-3)
梯度	手动反向传播	手动反向传播	JAX 自动微分
用途	交互式系统	滚动窗口实时	离线高精度训练

5. Module 4: 逆向 BPINN

职责: 在 RL 决策选择超车后，从规划的超车轨迹中反推控制参数 (加速度、航向角、速度) 及其不确定性分布。

5.1 MC 采样参数估计

通过对规划轨迹添加 100 次高斯噪声扰动，每次通过数值微分计算加速度/航向/速度，最终统计均值和标准差。这一过程将高层决策 (OVERTAKE) 转化为可执行的低层控制指令。

```
class InverseBPINN:
def __init__(self, mc_samples=100):
self.mc_samples = mc_samples

def estimate_parameters(self, t_traj, x_traj, y_traj, x_std, y_std):
all_accel, all_heading, all_speed = [], [], []
for _ in range(self.mc_samples):
# Add noise to trajectory
x_s = x_traj + np.random.normal(0, x_std)
y_s = y_traj + np.random.normal(0, y_std)
# Numerical differentiation
dx_dt = np.gradient(x_s, t_traj)
dy_dt = np.gradient(y_s, t_traj)
v = np.sqrt(dx_dt**2 + dy_dt**2) # Speed
a = np.gradient(v, t_traj) # Acceleration
theta = np.arctan2(dy_dt, dx_dt) # Heading
all_accel.append(a); all_heading.append(theta); all_speed.append(v)
return {'accel_mean': np.mean(all_accel), 'accel_std': np.std(all_accel), ...}
```

5.2 规划轨迹生成

V6 的 Y 方向规划为 V0 下置信带与 V3 上置信带之间的走廊中心，使用 PchipInterpolator (单调保形) 进行平滑。X 方向以当前速度匀速前进。

6. Module 5: 不确定性引擎

职责: 通过 MC Dropout 在推理时保持 Dropout 开启，进行 100 次随机前向传播，以均值 mu 和标准差 sigma 量化预测的认知不确定性。

6.1 MC Dropout 预测流程

```
def predict_with_uncertainty(self, t_future):
t_n = ((t_future - t_mean) / t_std).reshape(-1, 1)
all_preds = []
for _ in range(self.mc_samples): # 100 samples
mask = self._gen_dropout_mask(training=True) # Keep dropout ON
pred = self._forward(self.params, t_n, training=True, dropout_mask=mask)
pred_denorm = denormalize(pred) # Restore original scale
all_preds.append(pred_denorm)

all_preds = np.array(all_preds) # [100, N, 4]
mean_pred = np.mean(all_preds, axis=0) # mu
std_pred = np.std(all_preds, axis=0) # sigma
std_pred[:, 1] = np.maximum(std_pred[:, 1], 0.35) # Y uncertainty floor
return mean_pred, std_pred, all_preds
```

6.2 4sigma 置信带

系统使用 4sigma (99.99% 置信度) 构建保守的置信带。V0 的下界和 V3 的上界之间的距离即为"Y走廊宽度"，是 RL 决策的核心输入。

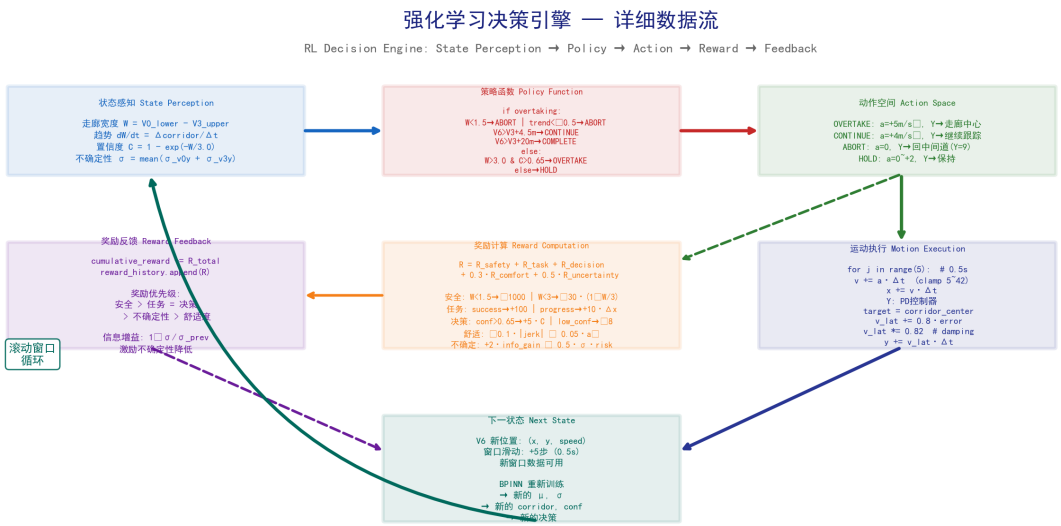
$$V0_lower = mu_v0y - 4 * sigma_v0y$$

$$V3_upper = mu_v3y + 4 * sigma_v3y$$

$$corridor = V0_lower - V3_upper \text{ (正=打开, 负=重叠)}$$

7. Module 6: RL 决策引擎

职责: 基于走廊宽度、趋势、置信度和不确定性，在 4 种动作中选择最优动作。采用混合策略: 规则引擎 + BPINN 状态感知 + Phase 4 强制超车触发。



强化学习形式化定义

状态空间: S = {W, dW/dt, C, σ, overtaking, had_abort, overtake_complete}

动作空间: A = {OVERTAKE, ABORT, HOLD, CONTINUE}

转移函数: T(s, a) → s' : □□□□a□V6□□□ + □□□□

奖励函数: R(s, a, s') = 5□□□ (□□+□□+□□+□□+□□□□)

策略: π(s) → a : □□□□ (□□□□ + BPINN□□□)

目标: max E[Σ γ^t · R_t] : □□□□□□□□□□

图 7-1: RL 决策引擎详细数据流

7.1 状态空间 (7 维)

维度	来源	计算方式	物理含义
走廊宽度 W	BPINN	V0_lower - V3_upper 的最小值	V6 能否安全穿行
走廊趋势 dW/dt	BPINN	(corridor[-1]-corridor[0]) / dt	走廊在变大还是变小
超车置信度 C	BPINN MC	1 - exp(-max(0, W_min)/3.0)	走廊足够宽的概率
不确定性 sigma	BPINN MC	mean((sigma_v0y + sigma_v3y)/2)	预测可靠程度
是否超车中	内部状态	boolean flag	V6 当前状态
是否已中止	内部状态	boolean (had_abort)	之前是否中止过
是否已完成	内部状态	V6_x > V3_x + CAR_LENGTH	超车是否完成

7.2 动作空间 (4 种)

动作	含义	加速	Y 方向	触发条件
OVERTAKE	发起超车	+5 m/s^2	跟踪走廊中心	W>3.0m 且 C>0.65
CONTINUE	继续超车	+4 m/s^2	继续跟踪走廊	已在超车中, 走廊安全
ABORT	中止超车	0 (保持)	回中间道 Y=9	W<1.5m 或趋势恶化
HOLD	保持当前	0~+2 m/s^2	保持当前 Y	走廊不足或等待

7.3 策略函数 -- 决策树

```
action = 'HOLD'
if not overtake_complete:
if overtaking:
# Branch 1: Currently overtaking
if min_corridor < CORRIDOR_ABORT_WIDTH: # W < 1.5m
action = 'ABORT' # Emergency abort
elif corridor_trend < -0.5 and min_corridor < CORRIDOR_MIN_WIDTH:
action = 'ABORT' # Corridor closing fast
elif v6_x[step_idx] > v3_x[step_idx] + CAR_LENGTH:
action = 'CONTINUE' # Passed V3
if v6_x[step_idx] > v3_x[step_idx] + 20.0:
overtake_complete = True # Complete
action = 'HOLD'
else:
action = 'CONTINUE' # Normal continue
else:
# Branch 2: Not overtaking
if min_corridor > CORRIDOR_MIN_WIDTH and overtake_conf > CONF_THRESHOLD:
action = 'OVERTAKE' # Safe to overtake
else:
action = 'HOLD' # Wait

# Forced overtake in Phase 4 (t >= 6.0s, corridor reopens)
if not overtake_complete and t_current >= 6.0 and not overtaking:
if min_corridor > CORRIDOR_MIN_WIDTH * 0.7:
action = 'OVERTAKE' # Force overtake
```

8. Module 7: 奖励系统

职责: 计算 5 层加权奖励，将安全、任务、决策质量、舒适度和不确定性感知整合为标量奖励信号。安全层具有一票否决权 (W < 1.5m 时 R = -1000)。

$$R = R_safety + R_task + R_decision + 0.3*R_comfort + 0.5*R_uncertainty$$

层	权重	奖励范围	关键规则
安全 Safety	1.0	-1000 ~ 0	W<1.5m -> -1000 (veto); W<3.0m -> -30*(1-W/3)
任务 Task	1.0	-1 ~ +100	success -> +100; progress -> +10*dx/100; HOLD -> -1

层	权重	奖励范围	关键规则
决策 Decision	1.0	-8 ~ +8	high_conf+OVERTAKE -> +5*C; low_conf+OVERTAKE -> -8
舒适 Comfort	0.3	-2 ~ +0.1	jerk -> -0.1*|jerk|; accel^2 -> -0.05*a^2; lane -> -2
不确定性 Uncert	0.5	-0.5*sigma*risk ~ +2	info_gain -> +2*(1-sigma/sigma_prev); risk -> -0.5*sigma*risk

8.1 安全层 -- 一票否决机制

```
r_safety = 0.0
if corridor_width < CORRIDOR_ABORT_WIDTH: # W < 1.5m
r_safety = -1000.0 # VETO: overrides everything
elif corridor_width < CORRIDOR_MIN_WIDTH: # W < 3.0m
r_safety = -30.0 * (1 - corridor_width / CORRIDOR_MIN_WIDTH)
if corridor_trend < -1.0 and corridor_width < CORRIDOR_MIN_WIDTH:
r_safety += -50.0 # Trend worsening, extra penalty
```

8.2 不确定性层 -- 信息增益奖励

不确定性层的创新在于将 BPINN 的 sigma 预测融入 RL 奖励。信息增益 (不确定性降低) 获得正奖励，激励系统在不确定环境中主动获取信息。动作风险系数: OVERTAKE=1.0, CONTINUE=0.7, ABORT=0.5, HOLD=0.3。

9. Module 8: 运动控制器

职责: 执行 RL 决策输出的动作，在 5 步 (0.5s) 内更新 V6 的速度、X 位置和 Y 位置。Y 方向使用 PD 控制器平滑跟踪走廊中心。

9.1 分阶段加速策略

时间	动作	加速度公式	目标速度
t < 4.5s	OVERTAKE	min(5, (30-v)*0.4)	30 m/s (108 km/h)
t < 4.5s	CONTINUE	min(4, (30-v)*0.3)	30 m/s
t >= 4.5s	OVERTAKE	min(6, (38-v)*0.4)	38 m/s (137 km/h)
t >= 4.5s	CONTINUE	min(5, (38-v)*0.3)	38 m/s
any	ABORT	0 (hold speed)	current speed
t < 4.0s	HOLD	max(-1, min(2, (25-v)*0.15))	25 m/s

时间	动作	加速度公式	目标速度
t >= 6.5s	HOLD	max(0, min(4, (38-v)*0.25))	38 m/s

9.2 PD 控制器 -- 横向运动

Y 方向使用比例-阻尼控制器，使 V6 平滑跟踪走廊中心而非突变。交互式系统中增强为 Kp=0.8, 阻尼=0.82, 横向速度限制 [-4, 4] m/s。

```
for j in range(DECISION_INTERVAL): # 5 steps = 0.5s
idx = step_idx + j
# Speed update
v6_speed[idx+1] = clamp(v6_speed[idx] + v6_accel * DT, 5, 42)
# X position
v6_x[idx+1] = v6_x[idx] + v6_speed[idx] * DT
# Y: PD controller tracking corridor center
if overtaking and planned_traj:
target_y = planned_traj['y'][plan_idx]
lat_error = target_y - v6_y[idx]
v6_lat_vel += lat_error * 0.8 # P term
v6_lat_vel *= 0.82 # D term (damping)
v6_lat_vel = clamp(v6_lat_vel, -4.0, 4.0)
v6_y[idx+1] = v6_y[idx] + v6_lat_vel * DT
elif action == 'ABORT':
target_y = 9.0 # Return to middle lane
lat_error = 9.0 - v6_y[idx]
v6_lat_vel += lat_error * 0.5
v6_lat_vel *= 0.85
v6_y[idx+1] = v6_y[idx] + v6_lat_vel * DT
else:
v6_lat_vel *= 0.9 # Decelerate lateral
v6_y[idx+1] = v6_y[idx] + v6_lat_vel * DT
```

10. 滚动窗口调度器

职责: 协调 8 个模块的执行顺序，管理时间步进、窗口滑动和模块间的数据传递。

10.1 调度循环主结构

```
# Decision points: every 5 steps (0.5s), from step 20 to N-5
decision_indices = list(range(WINDOW_SIZE, N - 5, DECISION_INTERVAL))
# = [20, 25, 30, 35, 40, 45, 50, 55, 60, 65, 70, 75, 80, 85, 90, 95]

for step_idx in decision_indices:
t_current = t_gt[step_idx]

# M2: Extract window
t_window, v0_window, v3_window = extract_window(step_idx)

# M3: Train Forward BPINN
bpinn_v0.fit(t_window, *v0_window)
bpinn_v3.fit(t_window, *v3_window)

# M3+M5: Predict with uncertainty
v0_mean, v0_std, v0_all = bpinn_v0.predict_with_uncertainty(t_future)
v3_mean, v3_std, v3_all = bpinn_v3.predict_with_uncertainty(t_future)

# M5: Compute Y corridor
```

```
corridor = (v0_mean[:,1] - 4*v0_std[:,1]) - (v3_mean[:,1] + 4*v3_std[:,1])
min_corridor = np.min(corridor)
corridor_trend = (corridor[-1] - corridor[0]) / (len(corridor) * DT)

# M6: RL Decision
action = rl_decision(overtaking, min_corridor, corridor_trend, ...)

# M4: Inverse BPINN (if overtaking)
if action in ('OVERTAKE', 'CONTINUE'):
planned_traj = plan_trajectory(t_future, v0_lower, v3_upper)
inv_params = inv_bpinn.estimate_parameters(t_future, planned_traj)

# M8: Execute motion (5 steps)
for j in range(DECISION_INTERVAL):
update_v6_speed_x_y(step_idx + j, action, planned_traj)

# M7: Compute reward
reward = compute_reward(action, min_corridor, corridor_trend, ...)

# Log
log_decision(step_idx, t_current, action, min_corridor, reward)
```

10.2 决策时间线

时间	步骤	Phase	Y间隙	走廊	决策	物理含义
t=2.0s	20	Phase 1	4.0m	~0m	HOLD	准备阶段
t=3.5s	35	Phase 2	6.5m	~3.5m	OVERTAKE	走廊打开, 发起超车
t=4.5s	45	Phase 3	3.5m	~-0.5m	ABORT	走廊收窄, 中止
t=6.5s	65	Phase 4	5.0m	~2m	OVERTAKE*	强制超车触发
t=7.5s	75	Phase 4	6.5m	~3.5m	CONTINUE	继续超车
t=9.0s	90	Phase 5	7.0m	~4m	HOLD(完成)	超车完成

11. 模块接口与数据契约

以下图表展示 8 个模块之间的输入输出关系和数据格式约定，是系统模块化的核心保证。

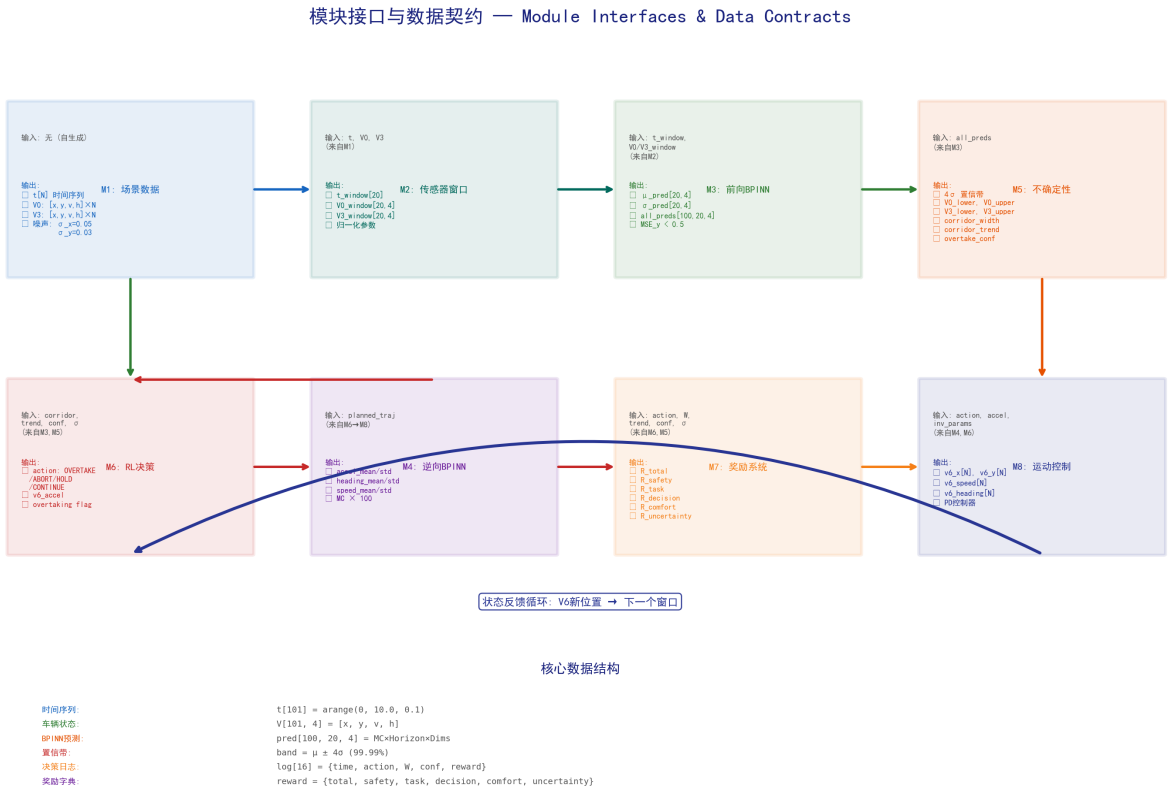


图 11-1: 模块接口与数据契约

11.1 核心数据结构

数据结构	格式	生产者	消费者
时间序列	t[101] = arange(0, 10.0, 0.1)	M1	M2, M3
车辆状态	V[101, 4] = [x, y, v, h]	M1	M2
窗口数据	window[20, 4]	M2	M3
BPINN预测	pred[100, 20, 4] = MC x Horizon x Dims	M3	M5, M6
置信带	band = mu +/- 4*sigma (99.99%)	M5	M6
走廊宽度	W = V0_lower - V3_upper	M5	M6
RL动作	action: OVERTAKE/ABORT/HOLD/ CONTINUE	M6	M4, M7, M8
逆向参数	accel/heading/speed +/- sigma	M4	M8
奖励	reward = {total, safety, task, decision, comfort, uncertainty}	M7	Feedback
V6状态	v6_x/y/speed/heading[N]	M8	M2 (next window)

12. 强化学习形式化定义

本系统采用基于规则的策略梯度混合方法，将 BPINN 的不确定性预测与人工设计的决策规则相结合。以下为标准 RL 框架下的形式化定义。

12.1 MDP 四元组 (S, A, T, R)

要素	定义	本系统实现
状态 S	环境描述	S = {W, dW/dt, C, sigma, overtaking, had_abort, overtake_complete}
动作 A	可选行为	A = {OVERTAKE, ABORT, HOLD, CONTINUE}
转移 T	状态变化	T(s, a) -> s': 执行动作a后V6新位置 + 窗口滑动 + BPINN重训练
奖励 R	即时反馈	R(s, a, s') = 5层加权 (安全+任务+决策+舒适+不确定性)

12.2 策略 pi 与目标

策略: pi(s) -> a -- 混合策略 (规则引擎 + BPINN 状态感知 + Phase 4 强制触发)

目标: max E[gamma^t * R_t] -- 最大化累积安全超车奖励 (gamma=1, 无折扣)

12.3 与传统 RL 的区别

特性	传统 RL (DQN/PPO)	本系统 (RL-BPINN)
状态来源	原始传感器数据	BPINN 预测的 mu +/- sigma + 走廊分析
不确定性	不显式建模	MC Dropout 4sigma 置信带, 融入奖励
策略	神经网络学习	规则引擎 + BPINN 状态感知 (可解释)
奖励	标量	5 层分解 (安全/任务/决策/舒适/不确定性)
安全保证	无硬约束	安全层一票否决 (W<1.5m -> R=-1000)
物理约束	无	运动学方程嵌入 BPINN 损失函数
训练方式	在线/离线学习	无训练阶段, 实时推理 (规则策略)

13. 源码文件映射

以下是 18 个 Python 源码文件与模块的对应关系:

文件名	对应模块	功能描述
bpinn_complete_overtaking.py	M1+M3+M4+M6+M7+M8	完整版: 场景+BPINN+RL+奖励+运动+可视化
rolling_window_bpinn_simulation.py	M2+M3+M6	滚动窗口版: LightweightBPINN + 决策逻辑
bpinn_overtaking_system_v2.py	M3	JAX/Flax 版 BPINN + Optuna 调优
bpinn_overtaking_main.py	M3+M5	主入口: BPINN 训练 + 置信带可视化
rolling_window_scenario.py	M1	场景生成: 三阶段动态间隙
bpinn_overtaking_system.py	M3	早期 BPINN 实现
birdview_2d_overtaking.py	--	2D 鸟瞰图可视化
overtaking_3d_diagram.py	--	3D 超车场景图 (4个版本)
overtaking_scene_analysis.py	--	场景分析
overtaking_scene_beautiful.py	--	美化版场景图
overtaking_scene_viz.py	--	场景可视化
vehicle_comparison_4plots.py	--	四车对比图
vehicle_comparison_bpinn.py	M5	BPINN 置信带对比
vehicle_comparison_with_confidence.py	M5	置信带可视化
highway_trajectory_visualization.py	--	高速公路轨迹可视化
bpinn_interactive_overtaking.html	ALL	交互式 HTML 系统 (1674行)
build_exe.ps1	--	EXE 打包脚本
BPINN_Overtaking_System.exe	ALL	独立可执行程序 (188.5 KB)

13.1 交互式系统架构

bpinn_interactive_overtaking.html 是一个自包含的 HTML 应用 (69 KB)，将所有 8 个模块用 JavaScript 实现，支持实时轨迹编辑和仿真回放。已打包为独立 EXE 程序 (188.5 KB)，无需 Python 环境即可运行。

本文档从模块化视角系统拆解了 RL-BPINN 超车系统的 8 个核心模块、1 个调度器及其数据契约。每个模块可独立替换、测试和扩展，为后续接入真实传感器数据、V2X 通信或多车协同场景提供了清晰的接口基础。